AGENTIC AI @ UIUC

From LLMs to Autonomous Engineers

CLAUDE CODE

Claude Code + Ralph Wiggum Loop + Super Ralph

Claude Code Workshop · April 2026

AGENDA

What We'll Cover

01

The Problem

Why LLMs alone fail at real engineering tasks

02

Claude Code

Architecture, agent loop, tools, memory & orchestration

03

Ralph Wiggum Loop

Clean-context iteration & spec-driven development

04

Super Ralph

Judge system, failure recovery & autonomous pipelines

05

What It Means

Claude Code vs Super Ralph & the road ahead

AXIOS

Mar 31, 2026 - Business

Anthropic leaked 500,000 lines o its own source code

Madison Mills

Codex announcement, plugins

vb Leader 6 3d

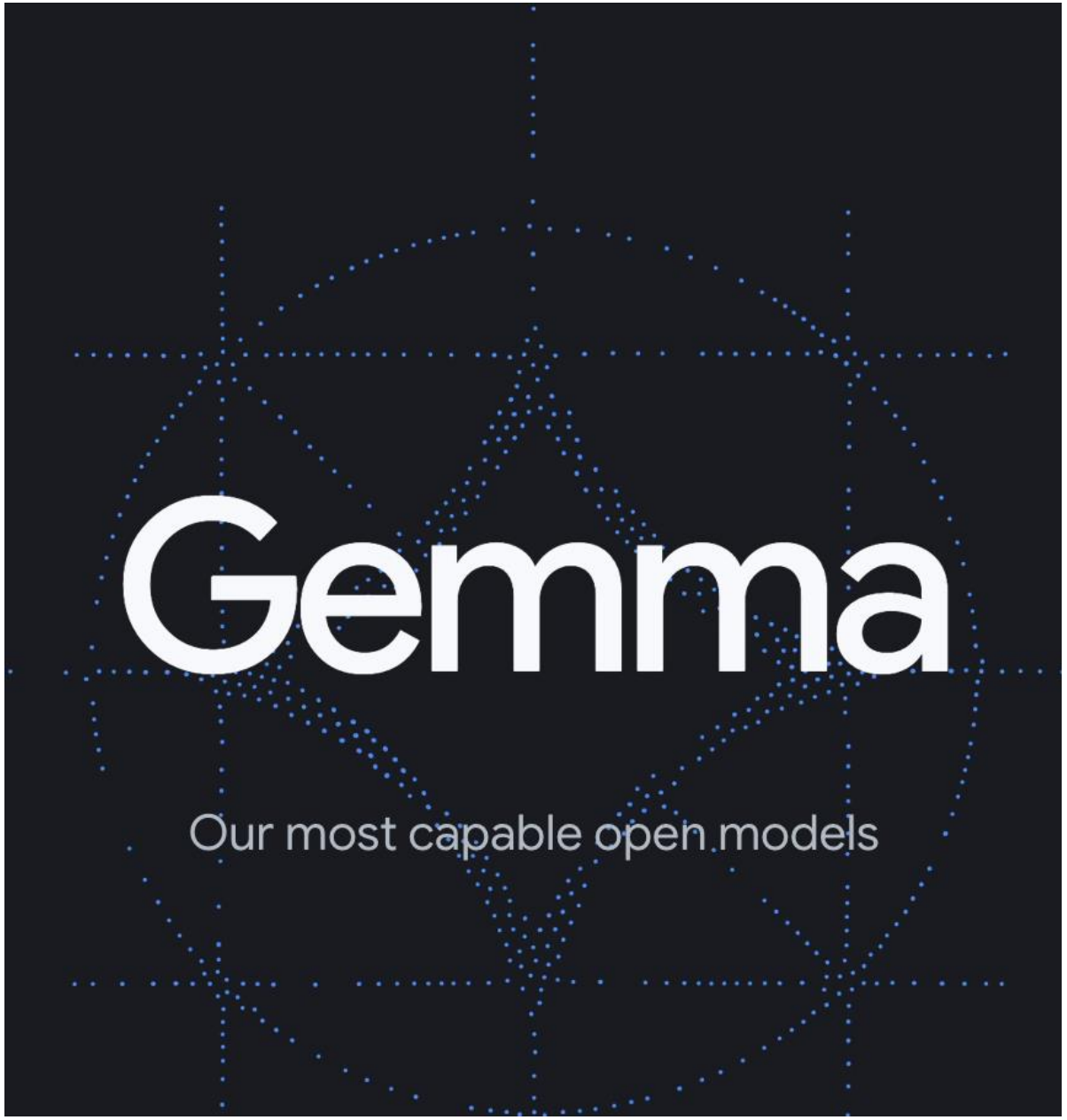
If you already use Claude Code, this Codex plugin is a simple way to bring Codex into the same workflow.

It is useful in three cases: a standard Codex review, a more skeptical adversarial review, and handing work off to Codex when you want a second pass from a different agent.

Check out the plugin here: GitHub - openai/codex-plugin-cc: Use Codex from Claude Code to review code or delegate tasks. · GitHub 8.0k

You need a ChatGPT subscription, including Free, or an OpenAI API key, plus Node.js 18.18 or later.

How it works:
The plugin delegates through the local Codex CLI and Codex app server. So it uses the same



claurst Public

Watch 106 | Fork 7.2k | Star

main | 1 Branch | 0 Tags | Go to file t | Add file | Code

Kuberwastaken feat: substantial revisions and progess, version cc d024edc · 39 minutes ago 5 Commits

public	rust port overhaul	yesterday
spec	rust port overhaul	yesterday
src-rust	feat: substantial revisions and progess, version cc	39 minutes ago
README.md	rust port overhaul	yesterday

README

IMPORTANT NOTICE - UPDATE

This repository does not hold a copy of the proprietary Claude Code typescript source code. This is a clean-room Rust reimplementation of Claude Code's behavior.

The process was explicitly two-phase:

Specification `spec/` - An AI agent analyzed the source and produced exhaustive behavioral specifications and

About

Your favorite Terminal Coc
now in Rust & a Breakdow
Claude Code leak & disco

kuber.studio/blog/AI/Cla

Readme
Activity
7.3k stars
106 watching
7.2k forks
Report repository

Contributors 1

Kuberwastaken Kuber

Languages

Rust 100.0%

PROBLEM

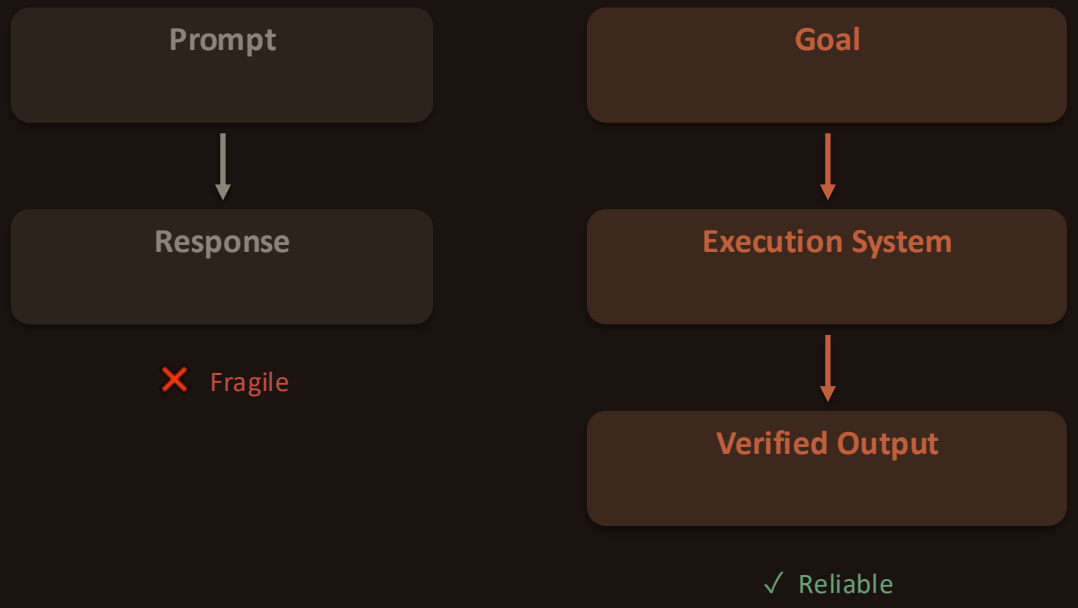
The Problem with LLMs Today

One-shot responses are brittle and context-blind

No iteration, no persistence across steps

Zero execution guarantees or verification

Fail systematically on long-horizon tasks



→ "Prompt + pray" does not scale to production workloads

PARADIGM SHIFT

From Models to Systems

LLM ≠ product — it's one component

The system around the model defines capability

Emergence of "agent runtimes" as a new category

Infrastructure > raw intelligence for production use

→ "The system, not the model, is the product"

CLAUDE CODE

What Claude Code Actually Is

Autonomous Agent Runtime

Not a chatbot — an execution engine
Runs in terminal, operates on real code

Reasoning + Tools + Loops

Combines LLM inference with tool calls
Iterates until task completion

Designed for Execution

Writes, tests, deploys, debugs
Goal-oriented, not conversation-oriented

HARNESS

= the control system wrapped around the model that defines what it can do, how it reasons, and when it stops

```
→  Claude Code is a harness, not a model
```

ARCHITECTURE

Claude Code Harness Architecture

→ Four components: Loop · Tools · Memory · Runtime

CORE PRIMITIVE

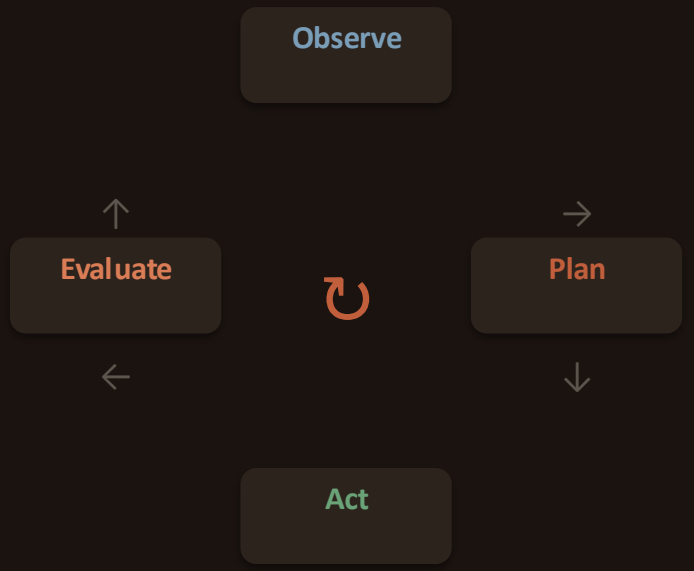
The Agent Loop

Iterative execution until goal completion

State evolves each step — not static

Not bounded to a single LLM response

Replaces prompting with iteration



→ Iteration replaces prompting

TOOLS

Tools + Constraints = Reliability

Structured tool calls with typed interfaces

Constrained action space prevents drift

Each tool is a well-defined contract

`read_file()`	Read file contents into context
`edit_file()`	Apply targeted edits to source
`run_tests()`	Execute test suite, parse results
`bash()`	Run shell commands in sandbox
`mcp_call()`	Invoke external MCP servers

→ Freedom → chaos · Constraints → reliability

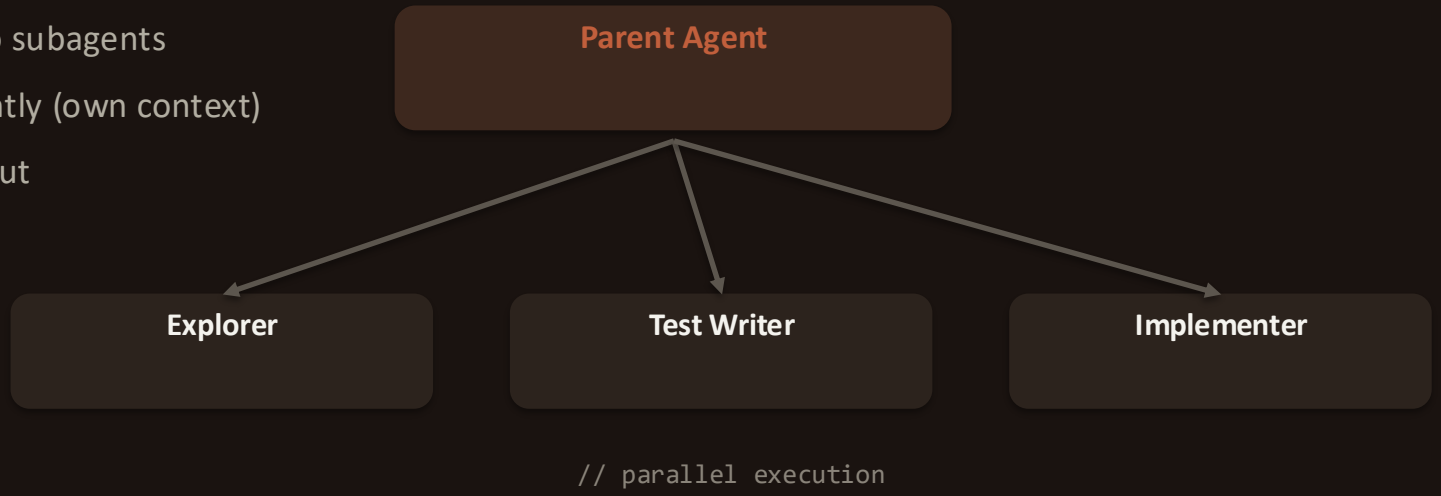
ORCHESTRATION

Multi-Agent Orchestration

Parent agent delegates tasks to subagents

Subagents operate independently (own context)

Parallel execution for throughput



→ Agents behave like distributed workers

MEMORY

Memory & Context Engineering

CLAUDE.md as persistent project instructions

AGENTS.md for multi-agent coordination

Selective retrieval over full context dump

Context reconstructed per step

→ Context is engineered, not static

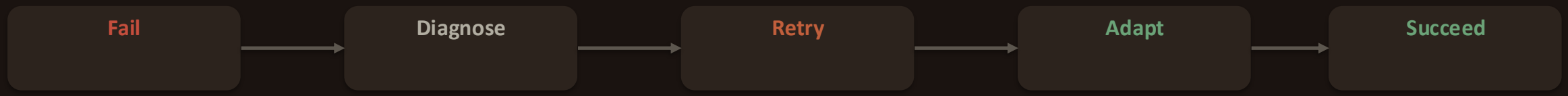
RELIABILITY

Failure Handling & Self-Correction

Built-in retry loops on failure

Self-evaluation (critic step) before committing

Alternate strategies when first approach fails



→ Reliability is designed, not hoped for

RALPH WIGGUM

The Ralph Wiggum Loop

Clean Context + Iteration

Each iteration runs in a clean context

State persisted externally (git, filesystem)

Prevents context drift & compounding errors

Stateless reasoning + stateful system

RALPH WIGGUM LOOP

1. Fresh Context
Clean slate each iteration

2. Read External State
git log, files, test results

3. Reason + Act
LLM plans and executes

4. Commit to External
git commit, write files

↻ repeat with clean context

→ "Memory outside the model, not inside it"

SPEC-DRIVEN

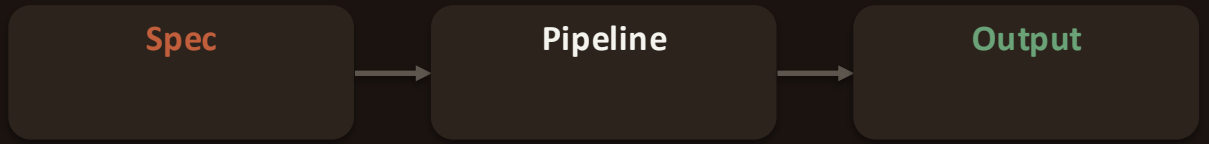
Spec-Driven Development

Specs define "done" — unambiguous success criteria

Tests enforce correctness automatically

Agent executes against structured requirements

Structured decomposition > open-ended prompts



Specification

Clear acceptance criteria
Structured task format
Measurable outcomes

Pipeline

Automated test execution
Build verification
Quality gates

Output

Verified, tested code
Passing test suite
Production-ready

→ AI performs better on structured decomposition than open-ended prompts

SUPER RALPH

Introducing Super Ralph

From Loop to Full Autonomous System

1)Starts with brainstorm/interview phase

2)Breaks tasks into structured subproblem

3)Assigns strict quality criteria per task

4)Independent subagents (no shared context)

5)Fully autonomous after initial setup

Interview
Gather requirements

Decompose
Break into subtasks

Execute
Independent subagents

Judge
Quality enforcement

Deliver
Verified output

→ Not just solving tasks — orchestrating systems

DEEP DIVE

Super Ralph Architecture

Task Decomposition
Breaks problem into subtasks

Subagent Execution
Independent workers

Test-First Pipeline
Tests before implementation

Judge Agent
Quality enforcement

Debugger Agent
Fallback recovery

Plan → Decompose → Execute → Judge → Retry

↩ retry loop

→ Separation of generation and evaluation

JUDGE

The Judge System

Every output evaluated against strict criteria

No "soft passes" — binary accept/reject

Unlimited retries until quality bar is met

REJECTS:

- ✕ TODO comments left in code
- ✕ Weak or trivial test cases
- ✕ Incomplete implementations
- ✕ Missing error handling
- ✕ Untested edge cases

JUDGE PIPELINE

Agent Output

↓

Criteria Check

↓

Test Execution

↓

Accept / Reject

↩ reject → retry

→ Quality is enforced, not assumed

ARCHITECTURE

Deterministic + Non-Deterministic Hybrid

DETERMINISTIC

- Pipelines & orchestration
- Test execution & validation
- Quality gates & criteria checks
- State management (git, fs)
- Retry logic & flow control

NON-DETERMINISTIC

- LLM reasoning & planning
- Code generation & refactoring
- Problem decomposition
- Error diagnosis & strategy
- Creative implementation choices

→ "The system is deterministic, the agent is creative"

RECOVERY

Failure Recovery System

3 failed attempts → debugger agent triggered

Fresh context analysis (no pollution from failures)

Generates independent fix strategy

Cold-start reasoning prevents local minima

→ Cold-start reasoning prevents local minima

COMPARISON

Claude Code vs Super Ralph

CLAUDE CODE

- General-purpose harness
- Flexible, adaptable to many tasks
- Less strict evaluation
- Interactive + autonomous modes
- Broad tool ecosystem (MCP)

SUPER RALPH

- Strict execution pipelines
- Enforced quality gates (Judge)
- Fully autonomous after setup
- Test-first development pipeline
- Production-oriented harness

→ Super Ralph = opinionated, production-oriented harness

IMPACT

Why This Changes Everything

From
Prompts
→
To
Pipelines

From
Tools
→
To
Systems

From
Coding
→
To
Orchestration

We are no longer writing software —
we are building systems that write software

Better models ≠ solution

Better harnesses = leverage

The future of software is agent infrastructure